SDN-Based Intelligent Handover and Load Balancing in Software-Defined Wireless Networks

Sara Eyad Kamal Hawash
An-Najah National University • s12217556@stu.najah.edu
Supervisor: Dr. Jihad Hamamreh

Abstract— This paper evaluates an SDN-assisted wireless handover and access-point (AP) selection prototype using Mininet-WiFi. The architecture separates OpenFlow-based Layer-2 forwarding, handled by an OS-Ken controller, from a Python management process that scans APs, reads station-count telemetry, computes policy decisions, and initiates reassociation. Three policies—strongest-RSSI, load-only, and a weighted RSSI/load policy—were compared over five repetitions each. Across 15 recorded runs, packet loss was zero. Load-only achieved the highest mean TCP receiver throughput (19.027 Mbps), the proposed policy achieved the lowest mean round-trip time (6.650 ms), and strongest-RSSI yielded the fewest mean TCP retransmissions. A separate observed handover run verified STA1 association to AP2 and successful connectivity. The results demonstrate prototype feasibility and metric-dependent trade-offs, while the small emulated topology and coarse load proxy limit generalization.

Index Terms— Software-defined networking, software-defined WLAN, handover, AP association, load balancing, RSSI, Mininet-WiFi.

I. Introduction

Wireless local-area networks often contain overlapping AP coverage areas. A client may select an AP based mainly on received signal strength indicator (RSSI), even when that AP has more associated stations than another available AP. This can create an association decision that favors signal strength but ignores a simple indicator of load. As the number of wireless clients and the diversity of traffic increase, AP association becomes relevant to throughput, delay, and user experience.

Software-defined networking (SDN) separates control logic from packet forwarding and enables programmable network management. In a software-defined WLAN, centralized or logically centralized management can collect network information and apply association policies. Mininet-WiFi provides an emulation environment for studying wireless SDN concepts and controller-based network behavior [1]. Prior work has examined handover and user-AP association management in SDN WLANs [2], [3], while QoS-aware load balancing has also been considered for dense software-defined Wi-Fi [4].

The objective of this project is to implement and evaluate an AP-selection decision that combines RSSI with the number of stations associated with each AP. The project compares a signal-only baseline, a load-only baseline, and a weighted policy. The evaluation is descriptive: it reports the recorded measurements and does not claim statistical significance or production-network performance.

II. Background and Related Work

In conventional WLAN association, the station and AP participate in a process influenced by signal quality, client behavior, and implementation-specific roaming logic. RSSI is useful as a measure of received signal level, but it does not directly indicate AP congestion. Two APs with different signal levels may have different numbers of associated stations and different traffic demand.

SDN-based WLAN management can provide a programmable point for association decisions. Mininet-WiFi has been used to emulate software-defined wireless networks and evaluate network-control approaches [1]. Research on terminal handover in SDN WLANs studies handover as a network-management problem rather than treating association as a purely local signal-strength choice [2]. User-AP association management is another related area, emphasizing that association policy can be coordinated through SDN mechanisms [3]. QoS-aware load balancing work further motivates considering load-related information when distributing clients across APs [4].

The present prototype uses a deliberately lightweight load indicator: the number of associated stations per AP. This indicator is easy to collect, but it is not equivalent to channel busy time, airtime utilization, queue occupancy, or per-client offered traffic. Therefore, the proposed policy should be interpreted as RSSI-plus-station-count selection, not as a complete radio-resource optimizer.

III. System Architecture

The emulated topology contains two APs and two stations. AP1 advertises SSID wifi1 on channel 1 (2412 MHz), and AP2 advertises SSID wifi2 on channel 6 (2437 MHz). STA1 is the managed wireless client; STA2 is used as the traffic endpoint. The topology is configured in Mininet-WiFi with a log-distance propagation model (exponent 3).

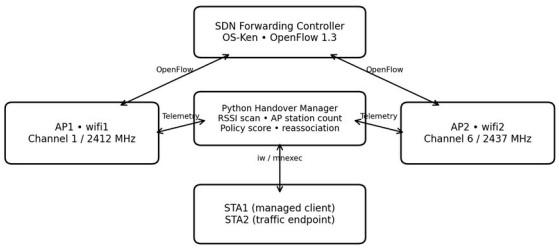


Fig. 1. Logical architecture of the emulated SDWN prototype.

The OS-Ken controller provides OpenFlow 1.3 Layer-2 MAC learning and flow installation for the AP forwarding plane. AP selection and handover are not implemented inside the OpenFlow controller. Instead, a separate Python management process performs wireless scanning, reads AP station-count data, calculates the selected policy, and invokes wireless commands in the STA1 network namespace. The manager uses the target AP SSID (wifi1 or wifi2) for reassociation and verifies the resulting association independently.

This separation is important when interpreting the architecture: the controller handles forwarding behavior, whereas the management process handles the association decision. The two components cooperate in the same emulated SDWN, but they do not represent a single integrated handover application running inside OS-Ken.

IV. AP Selection Policies

Three policies were evaluated. Strongest-RSSI selects the AP with the largest measured RSSI value. Load-only selects the AP with the smallest associated-station count. The proposed policy normalizes

RSSI and station count into bounded scores and combines them with fixed weights.

For AP i, the RSSI score is $R_i = \text{clamp}(2(r_i + 100), 0, 100)$, where r_i is RSSI in dBm. The load score is $Q_i = \text{clamp}(100 - 50L_i, 0, 100)$, where L_i is the number of stations associated with AP i. The combined score is $S_i = 0.6R_i + 0.4Q_i$. The proposed policy selects the AP with the largest S_i.

For the observed selection case, AP1 had RSSI −46 dBm and two associated stations, while AP2 had RSSI −60 dBm and zero associated stations. The corresponding RSSI scores were 100 and 80, and the load scores were 0 and 100. Thus, $S_1 = 0.6(100) + 0.4(0) = 60$, while $S_2 = 0.6(80) + 0.4(100) = 88$. The proposed policy therefore selected AP2 despite its weaker signal.

AP	RSSI	Stations	RSSI score	Load score	Weighted score
AP1	−46 dBm	2	100	0	60.0
AP2	−60 dBm	0	80	100	88.0

TABLE I. EXAMPLE INPUTS AND PROPOSED-POLICY SCORES

V. Handover Decision Procedure

The decision procedure first obtains a scan result from STA1 and extracts the visible AP SSIDs and RSSI values. It then reads the most recent station counts for AP1 and AP2. An AP is considered a candidate only when both its RSSI and load information are available. The selected policy is applied to the candidate set. In the proposed mode, the manager computes the weighted score for each candidate and chooses the AP with the maximum score.

After selecting a target, the manager disconnects STA1, waits briefly, and issues a connection command for the target SSID. The command return code is recorded, but it is not treated alone as proof of a successful handover. The link state is checked independently to verify the resulting SSID and BSSID. This verification step distinguishes command execution from confirmed association.

Algorithm 1 summarizes the proposed decision flow:

```
Input: AP scan results (RSSI), station counts Lᵢ, policy p
1: Build candidate AP set with valid RSSI and load data
2: For each candidate AP i:
3:    Rᵢ ← clamp(2(rᵢ + 100), 0, 100)
4:    Qᵢ ← clamp(100 − 50Lᵢ, 0, 100)
5:    Sᵢ ← 0.6Rᵢ + 0.4Qᵢ
6: If p = strongest-RSSI, select arg max rᵢ
7: If p = load-only, select arg min Lᵢ
8: If p = proposed, select arg max Sᵢ
9: Disconnect STA1 and connect to selected AP SSID
10: Verify association using wireless link-state output
Output: selected AP and verified association state
```

ALGORITHM 1. WEIGHTED RSSI/LOAD AP SELECTION AND HANDOVER

VI. Experimental Setup and Method

The documented environment used Ubuntu 24.04, Mininet-WiFi 2.7, OS-Ken 4.1.2, Python 3.12, and OpenFlow 1.3. The forwarding controller listened on TCP port 6653. The two APs used different Wi-Fi channels, and STA1 was returned to AP1 before each policy repetition. The management process then executed the selected policy.

The comparative dataset contains five repetitions for each policy, for 15 runs in total. Connectivity was measured with a 20-packet ICMP ping to 10.0.0.2. TCP throughput was measured using iperf3 for 10 seconds. The recorded metrics were receiver throughput, average RTT, packet loss, TCP retransmissions, and management-command duration. The wireless scan occasionally reported a busy device; the manager retried up to three times with a two-second wait.

Parameter	Recorded configuration
Emulation	Ubuntu 24.04; Mininet-WiFi 2.7
Controller	OS-Ken 4.1.2; OpenFlow 1.3; TCP 6653
Wireless topology	2 APs; 2 stations; STA1 managed
AP channels	AP1: ch. 1 / 2412 MHz; AP2: ch. 6 / 2437 MHz
Traffic tests	20-packet ping; 10-s iperf3 TCP
Repetitions	5 per policy; 15 comparative runs

TABLE II. EXPERIMENTAL CONFIGURATION

The handover_command_seconds field includes scanning, retry behavior, and association management. It is not a direct measurement of radio interruption, packet-service outage, or end-to-end handover latency. The station-count variable is a coarse load proxy; channel utilization was not established as a measured input in the comparative dataset.

VII. Results

Table III reports mean and sample standard deviation for each policy over five repetitions. Packet loss was zero in all 15 runs. Load-only achieved the highest mean TCP receiver throughput (19.027 Mbps), followed by Proposed (18.503 Mbps) and Strongest-RSSI (16.093 Mbps). Proposed recorded the lowest mean RTT (6.650 ms), while Strongest-RSSI recorded the fewest mean TCP retransmissions (279.6).

Policy	TCP Mbps	RTT ms
Strongest-RSSI	16.09 ± 0.18	7.00 ± 2.27
Load-only	19.03 ± 0.12	7.06 ± 0.98
Proposed	18.50 ± 0.92	6.65 ± 0.46

TABLE III. THROUGHPUT AND RTT (MEAN ± SAMPLE SD, n = 5)

Policy	Loss	Retransmissions	Cmd (s)
Strongest-RSSI	0%	279.6 ± 27.4	11.21 ± 0.02
Load-only	0%	380.0 ± 26.3	12.86 ± 0.94
Proposed	0%	370.8 ± 25.5	13.27 ± 0.03

TABLE IV. PACKET LOSS, RETRANSMISSIONS, AND COMMAND DURATION

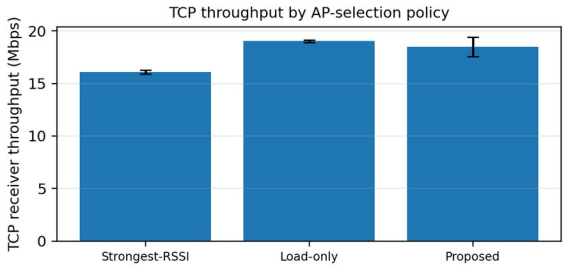


Fig. 2. TCP receiver throughput for all three policies; error bars show sample SD.

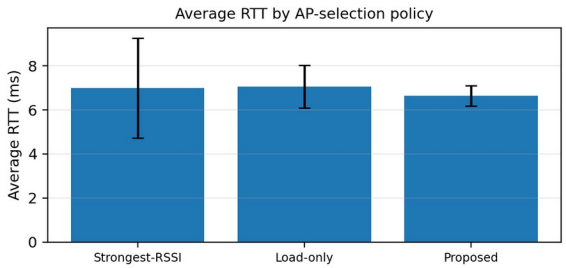


Fig. 3. Average RTT for all three policies; error bars show sample SD.

The mean management-command durations were 11.214 s for Strongest-RSSI, 12.858 s for Load-only, and 13.265 s for Proposed. These values should be interpreted cautiously because scanning and retries contribute to the measured duration. They should not be compared directly with the separate millisecond-scale connection-command observations from the earlier prototype log.

VIII. Separate Prototype Handover Observation

A separate observed Ubuntu 24.04 / Mininet-WiFi run documented STA1 initially associated with AP1. The observed RSSI values were

−46 dBm at AP1 and −60 dBm at AP2, while the station counts were two and zero, respectively. The proposed score selected AP2 (score 88 versus 60). The association was independently verified on SSID wifi2. The same run reported 19.1 Mbit/s TCP throughput and a 20-packet ping with 5.057 ms average RTT and zero packet loss.

This observation is separate from the 15-run policy-comparison dataset. It demonstrates that the reconstructed workflow selected AP2 and that connectivity was observed after the association change. It is not a paired before/after performance comparison and its throughput/RTT values are not included in Table III. A further single-scenario policy-decision log showed Strongest-RSSI selecting the already-current AP1, while Load-only and Proposed selected AP2; this observation is not sufficient to rank the policies statistically.

IX. Discussion

The results indicate that the preferred AP-selection policy depends on the performance objective. Load-only produced the highest average TCP throughput in the recorded comparative runs, suggesting that the station-count-based load choice was beneficial under this specific topology and traffic configuration. Proposed produced the lowest mean RTT, which indicates a different ranking when delay is considered. Strongest-RSSI produced the fewest mean TCP retransmissions. Therefore, the weighted policy should not be described as universally superior; its value is that it combines two indicators in a simple decision rule.

The proposed score uses fixed weights (0.6 for RSSI and 0.4 for load) and a linear station-count penalty. These values were used in the supplied implementation and were not optimized through a parameter search. A different topology, client distribution, or traffic mix may lead to a different ranking. In addition, station count treats all stations as equal, although real clients may have very different traffic demands and channel conditions.

The mean command duration was longest for Proposed and shortest for Strongest-RSSI in the comparative dataset. However, the command duration includes scan/retry overhead and is not equivalent to service interruption. A future evaluation should use continuous ping timestamps and packet gaps around reassociation to estimate interruption more directly. The supplied project scripts include ping-event analysis logic, but the current comparison table does not provide per-run interruption estimates.

X. Limitations and Threats to Validity

The experiment used only two APs and two stations in an emulated environment. Only one station was treated as the managed client, and station count was used as a coarse load indicator rather than measured airtime or channel utilization. The comparative dataset contains five repetitions per policy and the policy order was fixed rather than randomized. The scenario used a limited set of observed RSSI/load conditions; therefore, the results are descriptive and cannot establish general performance across arbitrary WLAN deployments.

The supplied project package identifies the code as a reconstructed development scaffold rather than the recovered original source repository. It also records that the scoped command `python -m pytest -q algorithms tests measurements` passed seven tests, while repository-wide pytest discovery was not clean because collection attempted to import a module requiring `pkg_resources`. These implementation notes are included for transparency and should not be interpreted as a complete integration-test pass.

Finally, channel utilization was not used as a verified comparative metric. The project contains a script that can calculate busy-time percentage when the wireless driver exposes valid active-time and busy-time counters. The documentation explicitly advises marking the metric unavailable when those counters are unsupported. Accordingly, this paper does not claim that station count equals channel utilization.

XI. Conclusion and Future Work

This paper presented an emulated SDWN prototype for AP selection and station handover. The architecture uses OS-Ken for OpenFlow 1.3 forwarding and a separate Python management process for scanning, load collection, policy scoring, and reassociation. In the 15-run comparison, Load-only achieved the highest mean TCP throughput, Proposed achieved the lowest mean RTT, and Strongest-RSSI produced the fewest mean TCP retransmissions; packet loss was zero in all runs. A separate observed run verified the proposed policy's AP2 selection and post-association connectivity.

Future work should expand the topology to more APs and stations, test multiple station positions and traffic profiles, randomize policy order, and target at least ten repetitions per scenario as suggested in the supplied experiment plan. It should also collect channel busy-time counters only when supported by the wireless driver, introduce hysteresis or a minimum dwell time to reduce unnecessary reassociation, and measure service interruption using continuous ping gaps. The weights in the proposed score could be studied under a broader scenario matrix, but any resulting performance claims should be supported by new controlled measurements.

Appendix A. Reproducibility Notes— To reproduce the documented emulation workflow, the project notes specify installing Mininet-WiFi and OS-Ken on Ubuntu, together with the wireless utilities `iw` and `iperf3`. The controller is launched first, followed by the Mininet-WiFi topology. The exact commands recorded in the supplied package are:

```
osken-manager --ofp-tcp-listen-port 6653
controller/l2_learning.py
sudo python3 topology.py
```

Before running a policy, the operator should confirm that both APs are visible, STA1 is associated, connectivity to the traffic endpoint is available, and RSSI and station-count telemetry can be read. Interface names can vary by Mininet-WiFi version; the supplied run notes explicitly recommend inspecting `iw dev` inside the relevant namespaces rather than assuming default interface names. The handover manager requires the current STA1 process identifier and executes wireless commands inside the station namespace using `mnexec`.

The recorded validation notes report seven passing tests for the scoped command `python -m pytest -q algorithms tests measurements`. The same notes state that repository-wide test discovery was not clean because collection attempted to import a module requiring `pkg_resources`. Accordingly, this report records the scoped test result and does not claim that the full repository test suite passed.

Appendix B. Additional Evaluation Plan

The supplied experiment plan proposes expanding the scenario matrix to two, four, and six stations where resources permit; placing STA1 near AP1, at the midpoint, and near AP2; and testing idle and loaded traffic profiles. It recommends at least ten repetitions per policy and scenario, with five as a minimum exploratory count. These are

planned future experiments, not results reported in the present evaluation.

Future records should include the selected AP, per-AP RSSI, station counts, association success, continuous-ping loss and maximum reply gap, iperf3 throughput, TCP retransmissions, and channel busy percentage only when valid active-time and busy-time counters are exposed by the wireless driver. Station count must remain labeled as a load proxy and must not be reported as measured channel utilization.

For a controlled follow-up, each policy should be tested under the same topology, station positions, traffic profile, and measurement duration. Policy order should be varied between repetitions where practical, and the run log should record the selected AP and association outcome together with the measurements listed above. This would make it easier to distinguish policy effects from changes in the emulated environment.

The expanded evaluation should report per-run observations as well as summary statistics. Throughput, RTT, retransmissions, and service interruption should be analyzed as separate outcomes; command execution time should remain a management-overhead measure. Any channel-busy result should be included only when the driver exposes valid counters, with the measurement method stated explicitly. These steps would support a more reproducible comparison without treating the present small dataset as conclusive.

References

[1] R. R. Fontes et al., “Mininet-WiFi: Emulating software-defined wireless networks,” in Proc. CNSM, 2015, pp. 384–389, doi: 10.1109/CNSM.2015.7367387.

[2] Z. Chen, Z. Luo, X. Duan, and L. Zhang, “Terminal handover in software-defined WLANs,” EURASIP Journal on Wireless Communications and Networking, vol. 2020, Art. no. 68, 2020, doi: 10.1186/s13638-020-01681-w.

[3] S. Bayhan, E. Coronado, R. Riggio, and A. Zubow, “User-AP association management in software-defined WLANs,” IEEE Transactions on Network and Service Management, vol. 17, no. 3, pp. 1838–1852, 2020, doi: 10.1109/TNSM.2020.3001869.

[4] S. Manzoor et al., “Towards QoS-Aware Load Balancing for High Density Software Defined Wi-Fi Networks,” IEEE Access, vol. 8, pp. 117623–117638, 2020, doi: 10.1109/ACCESS.2020.3004772.